

جامعة الجزائر 3
إبراهيم سلطان شيبوط



كلية العلوم الاقتصادية والعلوم التجارية وعلوم التسيير
قسم العلوم الاقتصادية

حل تطبيقات الأعمال الموجهة

"تطبيقات التعلم العميق"

تخصص: اقتصاد كمي "سنة أولى ماستر"

أستاذ المقياس:

د. حميدوش أحمد

إعداد الطالب(ة):

علواش سميرة

الكود الخاص بحل الأسئلة السبع "The code for solving the seven questions":

```
import tensorflow as tf
import numpy as np
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

plt.rcParams['font.family'] = 'DejaVu Sans'

# التمرين الأول: التعامل مع البيئة والتحقق منها

print("التمرين الأول: التعامل مع البيئة والتحقق منها")

print(f"\n1. المثبت TensorFlow إصدار: {tf.__version__}")

try:
    print(f"2. إصدار Keras (via tf.keras): {tf.keras.__version__}")
except AttributeError:
    print(f"2. إصدار Keras: مدمج ضمن TensorFlow (Keras 2.x أو 3.x)")

gpus = tf.config.list_physical_devices('GPU')
gpu_available = len(gpus) > 0
```

```
print(f"\n3. GPU متاحة: {gpu_available}")
```

```
if gpu_available:
```

```
    print(" tf.config.list_physical_devices('GPU') باستخدام الاستنتاج:
```

```
    print(f" GPU: {len(gpus)} عدد وحدات
```

```
else:
```

```
    print(" GPU متاحة الاستنتاج: القائمة فارغة، لا توجد
```

```
print("\n4. عند تدريب نموذج بسيط GPU تأثير غياب
```

```
print(" فقط (CPU) ، يتم التدريب باستخدام المعالج المركزي GPU عندما لا تتوفر
```

```
print(" النموذج البسيط مثل الانحدار الخطي لا يحتاج إلى عمليات معقدة أو متوازية
```

```
print(" سيكون صغيراً وغير ملحوظ GPU و CPU لذلك، الفرق في وقت التدريب بين
```

```
print(" لا يمنع تنفيذ النموذج، لكنه يبطئ التدريب في المهام الأكبر حجماً GPU غياب
```

```
# التمرين الثاني: التحليل النقدي لنتيجة التدريب
```

```
print("التمرين الثاني: التحليل النقدي لنتيجة التدريب")
```

```
print("\n1. (متوسط مربع الخطأ)؟ أعطِ صيغته الرياضية MSE ما المقصود بـ
```

```
print(" صيغته الرياضية Mean Squared Error. اختصار لـ MSE
```

```
print(" MSE = (1/n) * Σ (y_i الحقيقية - y_i المتوقعة)^2
```

```
print(" يقيس متوسط مربعات الأخطاء بين القيم الحقيقية والمتوقعة، وكلما صغر كانت دقة النموذج أفضل.")
```

```
print("\n2. مرتفعة بشكل غير  $MSE = 2.317$  في سياق بيانات مطبوعة بين 0 و 1، لماذا تعتبر قيمة 2.317 مرتفعة بشكل غير طبيعي؟")
```

```
print(" لأن البيانات محصورة بين 0 و 1، فإن أقصى فرق بين قيمة حقيقية وقيمة متوقعة هو 1")
```

```
print(" وبالتالي أكبر قيمة ممكنة لمربع الفرق هي 1، وأكبر قيمة لمتوسط هذه المربعات هي 1 أيضاً")
```

```
print(" أكبر من 1، وهذا مستحيل في الظروف الطبيعية  $MSE = 2.317$  لذلك فإن")
```

```
print(" يدل ذلك على وجود خطأ جسيم مثل عدم تطبيع البيانات أو تباعد النموذج بسبب معدل تعلم كبير جداً")
```

```
print("\n3. بالنسبة لنموذج تدرب جيداً على هذه البيانات؟ MSE ما القيمة التي تعتبرها مقبولة لـ")
```

```
print(" نموذج ممتاز: أقل من 0.01")
```

```
print(" نموذج جيد: بين 0.01 و 0.05")
```

```
print(" نموذج مقبول: بين 0.05 و 0.1")
```

```
print(" نموذج ضعيف: أكبر من 0.1")
```

```
print("\n4. صغ فرضية حول الأسباب المحتملة لهذه القيمة المرتفعة قبل أي تعديل على البرنامج")
```

```
print(" الفرضية الأولى: البيانات لم تُطبع (التطبيع لم يطبق أو طبق بشكل خاطئ)")
```

```
print(" كبير جداً مما يسبب تباعد النموذج (learning rate) الفرضية الثانية: معدل التعلم")
```

```
print(" غير كافٍ لوصول النموذج إلى التقارب (epochs) الفرضية الثالثة: عدد الحلقات")
```

```
print(" الفرضية الرابعة: التهيئة العشوائية للأوزان كانت سيئة")
```

```
# تحضير البيانات المستخدمة لبقية التمارين
```

```
print("تحضير البيانات لنماذج الانحدار الخطي")
```

```
np.random.seed(42)

surface = np.array([50, 60, 70, 80, 90, 100, 110, 120, 130,
140]).reshape(-1, 1)

prix = 2 * surface.flatten() + np.random.normal(0, 0.5, 10) # prix = 2 *
surface + noise
```

```
# التقييس (Standardization)
```

```
scaler_X = StandardScaler()
```

```
scaler_y = StandardScaler()
```

```
surface_norm = scaler_X.fit_transform(surface)
```

```
prix_norm = scaler_y.fit_transform(prix.reshape(-1, 1)).flatten()
```

```
mean_surface = scaler_X.mean_[0]
```

```
std_surface = scaler_X.scale_[0]
```

```
mean_prix = scaler_y.mean_[0]
```

```
std_prix = scaler_y.scale_[0]
```

```
print(f'\nعدد عينات التدريب: {len(surface)}')
```

```
print(f'متوسط المساحة: {mean_surface:.2f}, انحرافها: {std_surface:.2f}')
```

```
print(f'متوسط السعر: {mean_prix:.2f}, انحرافه: {std_prix:.2f}')
```

```
# التمرين الثالث: استخراج معادلة النموذج (باستخدام النموذج المحسّن)
```

```
print("التمرين الثالث: استخراج معادلة النموذج (بعد التصحيح)")
```

```

# بناء النموذج الجيد (LR=0.1)

model_good = tf.keras.Sequential([tf.keras.layers.Dense(1,
input_shape=(1,))])

optimizer_good = tf.keras.optimizers.Adam(learning_rate=0.1)

model_good.compile(optimizer=optimizer_good, loss='mse')

print("\n...جاري تدريب النموذج بمعدل تعلم 0.1 و 500 حقبة")

history_good = model_good.fit(surface_norm, prix_norm, epochs=500,
verbose=0, validation_split=0.2)

loss_good = history_good.history['loss']

val_loss_good = history_good.history['val_loss']

# استخراج الأوزان

weights_g, bias_g = model_good.get_weights()

w_g = weights_g[0][0]

b_g = bias_g[0]

# إلغاء التقييس

w_real_g = w_g * std_prix / std_surface

b_real_g = mean_prix + std_prix * b_g - w_g * std_prix * mean_surface /
std_surface

print("\n1. الأوزان والانحياز المستخرجة من النموذج المحسّن.")

```

```
print(f"  w (في الفضاء المقيس) = {w_g:.6f},  b = {b_g:.6f}")  
print(f"  المعادلة في الفضاء المقيس: prix_norm = {w_g:.6f} * surface_norm +  
{b_g:.6f}")
```

```
print("\n2. خطوات إلغاء التقييس:")  
print(f"  الميل الحقيقي = w * (std_prix / std_surface) = {w_g:.6f} *  
({std_prix:.2f} / {std_surface:.2f}) = {w_real_g:.6f}")  
print(f"  التقاطع الحقيقي = {mean_prix:.2f} + {std_prix:.2f} * {b_g:.6f} -  
{w_g:.6f} * {std_prix:.2f} * {mean_surface:.2f} / {std_surface:.2f} =  
{b_real_g:.6f}")
```

```
print("\n3. معادلة النموذج في الحجم الحقيقي:")  
print(f"  prix = {w_real_g:.6f} * surface + {b_real_g:.6f}")
```

```
# التحقق لمساحة 100  
surface_test = 100  
prix_manuel = w_real_g * surface_test + b_real_g  
print(f"\n4. {surface_test} متر مربع: التحقق: السعر المتوقع لمساحة =  
{prix_manuel:.2f}")
```

```
# التمرين الرابع: تشخيص مشكلة التقارب
```

```
print("التمرين الرابع: تشخيص مشكلة التقارب")
```

```
print("\n1. تحليل النتائج")
print("التوقعات: [-0.1458, -0.2918, -0.3892, -0.4866, -0.6320]")
print("القيم الحقيقية: [0.3333, 0.5333, 0.6667, 0.8000, 1.0000]")
print("جميع التوقعات سالبة بينما القيم الحقيقية موجبة ومحصورة بين 0 و 1")
print("هذا يعني أن النموذج فشل في التعلم وتباعد، والسبب هو أن معدل التعلم كان كبيراً جداً")
```

```
print("\n2. المعاملان المسؤولان عن هذا الخلل في البرنامج الأولي")
print("إذا تم ضبطه بقيمة كبيرة جداً (مثل 0.5 أو 1.0) (Learning Rate) معدل التعلم 1")
print("إذا كانت قيمها كبيرة جداً (Initial Weights) الأوزان الأولية 2")
```

```
print("\n3. العلاقة بين معدل التعلم وسرعة تقارب الشبكة العصبية")
print("معدل تعلم صغير جداً: تقارب بطيء جداً -")
print("معدل تعلم مناسب (0.001-0.01): تقارب سريع ومستقر -")
print("معدل تعلم كبير جداً: يسبب تباعد النموذج وعدم التقارب -")
print("الخلاصة: زيادة معدل التعلم تسرع التقارب ولكن بعد حد معين تسبب عدم استقرار")
```

```
print("\n4. ما المقصود بتقارب النموذج؟ كيف تكتشفه بدالة الخسارة؟")
print("تقارب النموذج يعني وصول دالة الخسارة إلى قيمة شبه ثابتة لا تتحسن كثيراً مع مزيد من التدريب")
print("، فإن (e-6 أقل من 1) يُكتشف بمراقبة قيم الخسارة كل حقبة: عندما تصبح التغيرات صغيرة جداً ، النموذج تقارب")
```

التمرين الخامس: تصحيح البرنامج وتحسينه - مع رسم منحنيات متعددة للمقارنة


```
print("LR=0.001 و LR=0.1 التمرين الخامس: تصحيح البرنامج وتحسينه - المقارنة بين")
```

```
# للمقارنة (LR=0.001) تدريب نموذج بمعدل تعلم افتراضي
```

```
print("\n...جاري تدريب النموذج بمعدل تعلم 0.001 (الافتراضي) و 500 حقبة للمقارنة")
```

```
model_default = tf.keras.Sequential([tf.keras.layers.Dense(1,  
input_shape=(1,))])
```

```
optimizer_default = tf.keras.optimizers.Adam(learning_rate=0.001)
```

```
model_default.compile(optimizer=optimizer_default, loss='mse')
```

```
history_default = model_default.fit(surface_norm, prix_norm,  
epochs=500, verbose=0, validation_split=0.2)
```

```
loss_default = history_default.history['loss']
```

```
val_loss_default = history_default.history['val_loss']
```

```
# عرض الخسارة كل 100 حقبة للنموذج المحسّن
```

```
print("\n(LR=0.1): عرض الخسارة كل 100 حقبة للنموذج المحسّن")
```

```
for epoch in [100, 200, 300, 400, 500]:
```

```
    print(f"الحقبة {epoch:3d}: loss = {loss_good[epoch-1]:.6f}, val_loss =  
{val_loss_good[epoch-1]:.6f}")
```

```
# للتدريب بين النموذجين loss الرسم البياني الأول: مقارنة
```

```
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
```

```
plt.plot(range(1, 501), loss_default, label='LR=0.001 (Training)',
color='red', linestyle='--')

plt.plot(range(1, 501), loss_good, label='LR=0.1 (Training)',
color='blue')

plt.xlabel('Epoch')

plt.ylabel('MSE Loss')

plt.title('Comparison of Training Loss')

plt.legend()

plt.grid(True, linestyle=':', alpha=0.6)
```

بين النموذجين validation loss الرسم البياني الثاني: مقارنة

```
plt.subplot(1, 2, 2)

plt.plot(range(1, 501), val_loss_default, label='LR=0.001 (Validation)',
color='red', linestyle='--')

plt.plot(range(1, 501), val_loss_good, label='LR=0.1 (Validation)',
color='blue')

plt.xlabel('Epoch')

plt.ylabel('Validation Loss')

plt.title('Comparison of Validation Loss')

plt.legend()

plt.grid(True, linestyle=':', alpha=0.6)

plt.tight_layout()

plt.show()
```

```
# للنموذج المحسن فقط val_loss و loss: الرسم البياني الثالث
plt.figure(figsize=(8, 5))

plt.plot(range(1, 501), loss_good, label='Training Loss (LR=0.1)',
color='green')

plt.plot(range(1, 501), val_loss_good, label='Validation Loss (LR=0.1)',
color='orange')

plt.xlabel('Epoch')

plt.ylabel('Loss')

plt.title('Training and Validation Loss for Corrected Model (LR=0.1)')

plt.legend()

plt.grid(True)

plt.show()
```

```
print("\nالتعليق على تطور الخسارة:")

print(f"الخسارة انخفضت من: {loss_default[0]:.6f} (LR=0.001) النموذج الافتراضي - إلى {loss_default[-1]:.6f}." )

print(f"الخسارة انخفضت من: {loss_good[0]:.6f} (LR=0.1) النموذج المحسن - إلى {loss_good[-1]:.6f}." )

print("نلاحظ أن النموذج المحسن يتقارب أسرع بكثير ويصل إلى خسارة أقل - ")

print("الرسم البياني يوضح الفرق الكبير في سرعة التقارب بين المعدلين - ")
```

التمرين السادس: تفسير النتائج النهائية

```
print("التمرين السادس: تفسير النتائج النهائية")
```

```
print("\n1. تفسير المعادلة النهائية (بعد التصحيح): prix = {:.6f} * surface +  
{:.6f}".format(w_real_g, b_real_g))
```

```
print("{:.4f} يعني أن كل متر مربع يزيد السعر بمقدار {:.4f} = رياضياً: الميل  
{:.4f}".format(w_real_g, w_real_g))
```

```
print("منطقياً (عقاري): السعر يعتمد فقط على المساحة بسعر ثابت للمتر المربع، ولا توجد تكلفة ثابتة.  
").format(w_real_g, w_real_g))
```

```
print("\n2. قيمة الخسارة النهائية (للمودج المحسن) على التدريب:  
{:.8f}".format(loss_good[-1]))
```

```
print("{:.8f}".format(val_loss_good[-1]))
```

```
print("هل هذه القيم مرغوبة؟")
```

```
print("الخسارة على التدريب أصبحت قريبة جداً من الصفر، ولكن على التحقق هناك خطأ بسيط -")
```

```
print("هذا أفضل من الصفر المطلق لأنه يعمم بشكل أفضل. ولكن في التطبيقات الحقيقية، -")
```

```
print("وجود خطأ بسيط طبيعي، والصفر المطلق مريب ويدل على فرط تخصيص")
```

```
print("\n3. حدود هذا النموذج في بيانات عقارية حقيقية:")
```

```
print("يعتمد على المساحة فقط ويتجاهل عوامل مهمة كالموقع، عدد الغرف، الطابق، العمر -")
```

```
print("يفترض علاقة خطية بسيطة، بينما العلاقة الحقيقية غير خطية -")
```

```
print("لا يتعامل مع القيم الشاذة ولا يتكيف مع تغيرات السوق -")
```

```
print("\n4. اقتراح تحسينين")
```

```
print("إضافة ميزات أخرى (عدد الغرف، الموقع، العمر) واستخدام انحدار خطي متعدد -")
```

```
print("استخدام شبكة عصبية ضحلة بطبقة خفية (عدة خلايا) لالتقاط الالخطية - ")
```

التمرين السابع: التوثيق والتفكير النقدي (تقرير موجز)

```
print("التمرين السابع: التوثيق والتفكير النقدي - تقرير موجز")
```

```
print("""
```

:تلخيص المسار الكامل للتعلم خلال هذه الأعمال التطبيقية

1. وأهميته (Scaling) تطبيع البيانات:

.تطبيع البيانات هو عملية تحويل قيم الميزات إلى نطاق موحد (متوسط 0 وانحراف معياري 1)

.أهميته: يسرع التقارب، يمنع هيمنة ميزات ذات قيم كبيرة، ويحسن الاستقرار العددي

2. (التعريف والتفسير) MSE دالة الخسارة:

.متوسط مربعات الأخطاء. تقيس أداء النموذج، وكلما صغرت كان أفضل MSE

.ولكنها حساسة للقيم الشاذة

3. دور المعاملات الفائقة: معدل التعلم وعدد الحقب

.تباعد $\rightarrow$ تقارب بطيء، كبير جداً $\rightarrow$ معدل التعلم: يتحكم في حجم الخطوة. صغير جداً -

.فرط تخصيص $\rightarrow$ عدم التقارب، كثير جداً $\rightarrow$ عدد الحقب: عدد مرات التدريب. قليل جداً -

4. استخراج معاملات نموذج الانحدار الخطي:

:ثم نطبق معادلة إلغاء التقييس `model.get_weights()` نستخدم

$$w_real = w * (std_y / std_x)$$

$$b_real = mean_y + std_y * b - w * std_y * mean_x / std_x$$

5. تفسير التوقعات وتقييم الأداء:

.يجب استخدام بيانات اختبار منفصلة. النموذج الجيد يعمي. خسارة صفرية على التدريب غير مرغوبة

الخلاصة: قمنا بالتحقق من البيئة، حللنا النتائج، دربنا نموذجين، قارنا بين معدلات التعلم،

.استخرجنا المعادلات، وفهمنا حدود النموذج وأهمية التطبيع واختيار المعاملات الفائقة

)"""

